

Kiến trúc Navigation Stack:

Flutter quản lý các routes dưới dạng Stack và được quản lý bởi Navigation (bản chất là 1 StatefulWidget chuyên dụng)

***Navigation.push:** Thêm 1 route mới lên trên đỉnh của Stack =>

Màn hình mới sẽ đè lên màn hình cũ

-Màn hình cũ (màn hình 1) không bị destroy, nó vẫn tồn tại trong Element Tree nhưng tạm thời bị ẩn khỏi RenderObject và bị màn hình 2 che khuất

-Mỗi màn hình sẽ trải qua các bước: initState ->

didChangeDependencies -> build -> hiển thị

***Navigation.pop:** Xóa Route trên đỉnh Stack, đưa ứng dụng về màn hình ngay bên dưới

-Màn hình 2 sẽ bị xóa khỏi Stack (nghĩa là cả 3 cây Widget, Element, RenderObject sẽ gỡ bỏ hoàn toàn màn hình 2 unmount và giải phóng bộ nhớ

-Màn hình 2 sẽ gọi deactivate -> dispose

=====

Thực tế trong flutter sẽ dùng **MaterialPageRoute** => thực hiện tự động, tránh lạm dụng push quá nhiều (quên pop) => phình bộ nhớ RAM

=====

Cơ chế nhận dữ liệu phản hồi từ màn hình 2 về màn hình 1?????

Sử dụng cơ chế Future: Hàm Navigator.push() trả về 1 future (đại diện cho 1 giá trị chưa xuất hiện lại lúc runtime nhưng sẽ trả về trong tương lai khi màn hình 2 đóng lại

Sử dụng luồng xử lý (Async/Await):

Màn hình 1 tạm dừng và thực thi dòng lệnh: await Navigator.push và trả quyền điều khiển cho Event Loop của Dart Thread

Màn hình 2: khi thực hiện Navigator.pop(context, resultData) thì giá trị resultData sẽ được đẩy về và giải quyết cho Future mà màn hình 1 đang đợi

=====

Listview: